

GUÍA DE LABORATORIO AVANZADO: COMPUTACIÓN DISTRIBUIDA CON DASK, DOCKER & PREFECT

Procesamiento Out-of-Core de 300.000 Registros Masivos, Limpieza con Expresiones Regulares, Normalización de Encoding y Orquestación Multi-Nodo

Módulo / Contexto:	Patrones Arquitectónicos Avanzados / Sistemas Distribuidos y Big Data
Topología de Cómputo:	1 Scheduler Central + 3 Workers Dask independientes en red Docker Compose
Volumen de Datos:	300.000 filas con ruido sintético (mojibake UTF-8, valores nulos, regex caótico)
Stack Tecnológico:	Python 3.11+, Dask Distributed, Prefect 2.x/3.x, Docker & Docker Compose

1. Fundamentos Arquitectónicos y Objetivos de la Práctica

En la ingeniería de software contemporánea, el manejo de grandes volúmenes de datos colisiona recurrentemente con los límites físicos de memoria RAM de un único nodo de cómputo (problema conocido como Out-of-Memory / OOM). Las arquitecturas monolíticas en memoria (como Pandas estándar) requieren cargar la totalidad del dataset antes de ejecutar transformaciones vectorizadas, lo que resulta inviable cuando el volumen supera la capacidad del heap o cuando las transformaciones de limpieza implican parsing intensivo de cadenas y decodificación compleja.

Este taller implementa de manera práctica el Patrón Arquitectónico Master-Worker acoplado a un motor de Grafos Acíclicos Dirigidos (DAGs) mediante Dask, complementado con un orquestador de flujos de trabajo de grado empresarial (Prefect). Los estudiantes explorarán cómo una red de contenedores independientes colabora para resolver un cuello de botella analítico sobre un lote de 300.000 registros altamente corruptos.



PRINCIPIO DE DISEÑO ARQUITECTÓNICO

El patrón Master-Worker desvincula la coordinación del cómputo. El Scheduler (Master) se encarga exclusivamente de mantener el estado del grafo de tareas, optimizar la asignación de dependencias y balancear la carga de memoria. Los Workers ejecutan el trabajo físico sobre particiones de datos en memoria acotada y transfieren resultados parciales mediante comunicación directa Peer-to-Peer (P2P), evitando cuellos de botella en el nodo maestro.

Objetivos de Aprendizaje:

- **1. Despliegue de Infraestructura Distribuida:** Diseñar e inicializar una topología de clúster con 1 Scheduler y 3 Workers aislados en contenedores Docker mediante Docker Compose.
- **2. Computación Out-of-Core y Particionado:** Comprender la fragmentación de un dataset de 300.000 filas en 'chunks' manejables para su procesamiento asíncrono y paralelo.
- **3. Resiliencia y Limpieza de Calidad de Datos:** Diseñar funciones vectorizadas y mapeadas para reparar corrupciones de codificación (mojibake UTF-8/Latin-1) y extraer identificadores canónicos mediante Expresiones Regulares.
- **4. Refactorización de Columnas en Clúster:** Transformar una columna heterogénea y sucia en una estructura estandarizada con validación de anomalías a nivel distribuido.
- **5. Orquestación y Observabilidad:** Integrar un flujo reactivo con Prefect (@flow, @task) y monitorear el consumo de recursos en tiempo real en el Dask Dashboard.

2. Infraestructura del Clúster con Docker Compose

Para simular un entorno distribuido de producción sin requerir acceso a clústeres en la nube (AWS/GCP), utilizaremos Docker Compose. Cada nodo correrá en su propio contenedor Linux, compartiendo una red virtual tipo bridge ('dask-cluster-net') y un volumen local montado ('shared-data') que emulará un sistema de archivos distribuido o Data Lake (S3/GCS).

Topología de Red del Clúster:

Contenedor / Servicio	Rol en el Patrón	Puertos Expuestos	Recursos Asignados
dask-scheduler	Master / Coordinador	8786 (IPC), 8787 (Dashboard)	1 vCPU, 1 GB RAM
dask-worker-1	Worker Node 1	Interno (Red Docker)	2 Threads, 1.5 GB RAM
dask-worker-2	Worker Node 2	Interno (Red Docker)	2 Threads, 1.5 GB RAM
dask-worker-3	Worker Node 3	Interno (Red Docker)	2 Threads, 1.5 GB RAM

3. Generador de Datos Sintéticos Sucios (300.000 Filas)

En proyectos de Big Data reales, los datos crudos rara vez vienen sanitizados. Para poner a prueba la resiliencia de nuestro pipeline distribuido, crearemos un script generador (`generate_dirty_data.py`) que inyecta anomalías de forma probabilística a lo largo de 300.000 registros.

Anomalías Inyectadas en el Dataset:

- **1. Refactorización Objetivo ('raw_customer_code'):** Códigos desestructurados que mezclan prefijos, guiones, corchetes, espacios parásitos y casos atípicos (ej. ' CLI-98234-A ', 'id_98234_norm', 'RAW#98234-[V2]', 'INVALID_DATA', '', NaN). Debe refactorizarse a 'CUST-XXXXX'.

- **2. Errores de Codificación Mojibake ('city_notes_corrupted'):** Textos en español con tildes y caracteres especiales corrompidos intencionalmente mediante codificaciones cruzadas (UTF-8 interpretado erróneamente como ISO-8859-1 o Windows-1252, generando 'Bogotá', 'Medellán', 'Cancún', 'Operación').
- **3. Heterogeneidad en Teléfonos ('phone_raw'):** Números con formatos internacionales caóticos (+57, paréntesis, puntos, sufijos con extensiones 'Ext 104', cadenas vacías o 'DESCONOCIDO').
- **4. Valores Nulos y Espacios Vacíos:** Distribución estadística de nulos (NaN, None, y cadenas con espacios en blanco ' ') en múltiples columnas.

```
[PYTHON - generate_dirty_data.py]
#!/usr/bin/env python3
"""
generate_dirty_data.py
Generador sintético de 300.000 registros con anomalías severas de calidad de datos.
"""

import os
import random
import numpy as np
import pandas as pd

def generate_dirty_dataset(num_rows=300_000, output_dir="shared-data/raw"):
    os.makedirs(output_dir, exist_ok=True)
    print(f"[*] Iniciando generación de {num_rows:,} filas con ruido sintético...")

    # Semilla para reproducibilidad experimental
    np.random.seed(42)
    random.seed(42)

    # 1. Columna a refactorizar: raw_customer_code
    # Patrón esperado: Código numérico de 5 dígitos inmerso en ruido
    clean_ids = np.random.randint(10000, 99999, size=num_rows)
    templates = [
        " CLI-{id}-A ",
        "cli_{id}_norm",
        "RAW#{id}-[V2]",
        "CUST-{id}-EXP",
        " {id} ",
        "ANOMALOUS_STR",
        "INVALID",
        " ",
        None
    ]
    weights = [0.45, 0.20, 0.15, 0.10, 0.04, 0.02, 0.02, 0.01, 0.01]
    raw_customer_code = [
        random.choices(templates, weights=weights)[0].format(id=cid)
        if random.choices(templates, weights=weights)[0] is not None else np.nan
        for cid in clean_ids
    ]

    # 2. Columna con Mojibake y encoding corrupto
    mojibake_phrases = [
        "Transacción exitosa en Bogotá D.C.",
        "Cliente atendido en Medellín por garantía",
        "Verificación de crédito rechazada en Popayán",
        "Envío expreso hacia Cartagena de Indias",
    ]
```

```

    "Actualizaci3n de direcci3n en Cali (Valle)",
    "Operaci3n pendiente de conciliaci3n bancaria",
    "Sin observaciones registradas",
    "Ã±andÃ± importado - paquete especial",
    "  ",
    None
]
phrase_weights = [0.25, 0.20, 0.15, 0.15, 0.10, 0.08, 0.03, 0.02, 0.01, 0.01]
corrupted_notes = random.choices(mojibake_phrases, weights=phrase_weights, k=num_rows)

# 3. Columna de tel3fonos ca3ticos
phone_templates = [
    "+57 (310) {p1}-{p2}",
    "310.{p1}.{p2}",
    "TEL: 310{p1}{p2} Ext 402",
    "0057 310 {p1} {p2}",
    "310{p1}{p2}",
    "DESCONOCIDO",
    "N/A",
    "--",
    ""
]
phone_weights = [0.35, 0.25, 0.15, 0.10, 0.08, 0.03, 0.02, 0.01, 0.01]
raw_phones = [
    random.choices(phone_templates, weights=phone_weights)[0].format(
        p1=random.randint(100, 999), p2=random.randint(1000, 9999)
    )
    for _ in range(num_rows)
]

# 4. M3tricas num3ricas con valores nulos y formatos heterog3neos
amounts = np.random.exponential(scale=150.0, size=num_rows)
# Inyectar 3% de NaN
nan_mask = np.random.rand(num_rows) < 0.03
amounts[nan_mask] = np.nan

# 5. Timestamp y categor3a transaccional
categories = random.choices(["FINANCE", "LOGISTICS", "RETAIL", "HEALTH", "TECH"],
                             k=num_rows)

# Consolidar DataFrame maestro
df = pd.DataFrame({
    "transaction_id": [f"TX-{i:07d}" for i in range(1, num_rows + 1)],
    "raw_customer_code": raw_customer_code,
    "city_notes_corrupted": corrupted_notes,
    "phone_raw": raw_phones,
    "amount_usd": np.round(amounts, 2),
    "business_category": categories
})

# Guardar particionado en 6 archivos CSV (~50.000 filas c/u)
# Esto facilita que los 3 workers reciban chunks balanceados
num_chunks = 6
chunk_size = num_rows // num_chunks
for idx in range(num_chunks):

```

```

chunk_df = df.iloc[idx * chunk_size : (idx + 1) * chunk_size]
chunk_path = os.path.join(output_dir, f"transactions_dirty_part_{idx+1}.csv")
chunk_df.to_csv(chunk_path, index=False, encoding="utf-8")
print(f" -> Generada partición {idx+1}/{num_chunks}: {chunk_path} ({len(chunk_df):,}
filas)")

print("[✓] Conjunto de datos sintético generado exitosamente en shared-data/raw/")

if __name__ == "__main__":
    generate_dirty_dataset()

```

4. Pipeline Distribuido de Limpieza y Refactorización

A diferencia de Pandas, donde las transformaciones se ejecutan de inmediato en un único hilo, en Dask trabajamos con DataFrames perezosos (`dask.dataframe`). Las operaciones de limpieza no tocan el disco ni la CPU hasta que se invoca el método `.compute()` o se escribe a un formato estructurado como `.to_parquet()`.

Estrategia de Refactorización de la Columna 'raw_customer_code':

La columna objetivo contiene identificadores con formatos heterogéneos generados por múltiples sistemas legados. El objetivo arquitectónico es mapear esta columna a un estándar canónico unificado `CUST-XXXXX`, aislando anomalías:

- **Extracción Regex:** Aplicar el patrón `r'(?<CLI-|cli_|RAW#|CUST-)?(\d{5})'` para extraer el número base de 5 dígitos.
- **Manejo de Casos Nulos o Inválidos:** Si la expresión regular no encuentra un identificador válido o la fila contenía nulos/vacíos, se imputa un token de alerta: `'CUST-00000-ANOMALY'`.
- **Corrección de Mojibake:** Utilizar una función de corrección de codificación que detecte secuencias típicas de decodificación errónea (UTF-8 interpretado como Latin-1) y las restaure a español neutro correcto (`'Bogotá'` -> `'Bogotá'`).



PATRÓN DE OPTIMIZACIÓN EN DASK

Al manipular cadenas en Dask, las operaciones vectorizadas de `ddf['col'].str` se distribuyen automáticamente entre los workers. Sin embargo, para lógica compleja o regex con bifurcaciones condicionales, el método `ddf.map_partitions()` permite ejecutar código optimizado de Pandas por bloques, reduciendo el overhead de serialización entre el Scheduler y los Workers.

5. Orquestación del Flujo con Prefect

En una arquitectura empresarial, el procesamiento en clúster no ocurre en scripts aislados; requiere de un orquestador que gestione dependencias de tareas, monitoree estados (Scheduled, Running, Completed, Failed), maneje reintentos automáticos ante fallas de red y publique métricas de observabilidad.

Utilizaremos Prefect para encapsular el ciclo de vida completo del laboratorio: desde la validación de infraestructura y verificación de datos crudos, hasta la ejecución del DAG en los workers de Dask y las pruebas de aserción de calidad.

6. Preguntas de Análisis y Reflexión Arquitectónica

Al finalizar la ejecución práctica, cada grupo de estudiantes deberá entregar un breve informe respondiendo las siguientes preguntas de diseño de sistemas distribuidos:

- **1. Cuellos de Botella de Red vs. CPU:** ¿Qué impacto tiene la transferencia de datos entre contenedores en comparación con el tiempo puro de CPU dedicado a las expresiones regulares? ¿En qué punto la serialización de red hace más lenta una arquitectura distribuida frente a un proceso mononodo?
- **2. Tolerancia a Fallos y Recomputación:** Si durante la mitad del procesamiento ejecutas `docker stop dask-worker-2`, ¿qué ocurre con las tareas que estaban asignadas a ese nodo? ¿Cómo utiliza Dask el DAG para reprogramar los chunks perdidos en los workers 1 y 3?
- **3. Ventajas de Parquet sobre CSV:** ¿Por qué el pipeline exporta el resultado final a Apache Parquet en lugar de concatenar un CSV de 300k filas? Explica la compresión columnar y el filtrado por predicados (predicate pushdown).
- **4. Separación de Responsabilidades (Prefect vs. Dask):** ¿Por qué no utilizar únicamente Dask para todo el flujo? ¿Qué valor arquitectónico aporta Prefect en términos de gobernanza, reintentos y trazabilidad de negocio?

7. Rúbrica de Evaluación del Taller

Criterio de Evaluación	Ponderación	Desempeño Esperado (Excelente)	Puntaje
Infraestructura Docker y Clúster	20%	Levantamiento correcto de 1 Scheduler y 3 Workers independientes en Docker Compose con límites de recursos.	1.0 / 5.0
Script Generador y Datos Sucios	20%	Generación efectiva de 300.000 filas con patrones patológicos (mojibake, nulos, heterogeneidad regex).	1.0 / 5.0
Refactorización y Limpieza Dask	35%	Implementación de transformaciones distribuidas eficientes, reparación de encoding y estandarización a CUST-XXXXX.	1.75 / 5.0
Orquestación Prefect y Análisis	25%	Integración de <code>@flow</code> y <code>@task</code> , validación de quality gates y respuestas fundamentadas a las preguntas arquitectónicas.	1.25 / 5.0

